

Event-Consistent Dual-Graph Mixture-of-Experts for Dynamic Flexible Job Shop Scheduling Under Multiple Disturbances

Hongyao Ma, *Graduate Student Member, IEEE*, Jing Bi, *Senior Member, IEEE*, Ziqi Wang, *Graduate Student Member, IEEE*, Jiahui Zhai, *Member, IEEE*, Junqi Zhang, *Senior Member, IEEE*, and MengChu Zhou, *Fellow, IEEE*

Abstract—Dynamic flexible job shop scheduling (DFJSP) under concurrent disturbances requires online dispatching policies that adapt across both production scales and disturbance regimes. Existing deep reinforcement learning schedulers have shown promise, but their robustness under simultaneous scale expansion and heterogeneous disturbances remains limited. This paper proposes E2G-MoE, an event-consistent dual-graph Mixture-of-Experts framework that combines a dispatching interface consistent with event-driven execution, a scalable dual-graph encoder, and a disturbance-conditioned hierarchical policy. The encoder decouples stable operation precedence from dynamic machine competition, while the policy routes candidate job-machine decisions according to the observed disturbance context. Experiments on synthetic instances and public benchmarks demonstrate competitive learning-based performance, stronger robustness in the evaluated large-scale and compound-disturbance settings, and favorable transfer across tested scales and benchmark families without retraining.

Index Terms—Dynamic flexible job shop scheduling, Deep reinforcement learning, Dual-graph representation, Mixture-of-Experts, Cross-scale transfer

I. INTRODUCTION

Modern manufacturing systems involve high product variety, small batches, and tightly coupled shared resources, resulting in continuously changing shop floors [1]. Dynamic flexible job shop scheduling (DFJSP) must therefore respond to event-triggered job insertions and machine failures/recoveries

while accounting for scenario-dependent transportation times that remain fixed within each rollout [2]. These scheduling decisions are naturally triggered at state-changing events rather than arbitrary time steps [4]. Compared with static FJSP, the dynamic setting changes not only operation release times but also machine availability, feasible job-machine pairs, and effective operation start times. A policy that performs well under one operating condition can therefore become suboptimal when the disturbance pattern changes, especially when several disturbances occur in the same production horizon.

Deep reinforcement learning (DRL) has advanced static scheduling, dynamic job arrivals, transportation-aware scheduling, and large-scale dispatching without handcrafted rule design [3], [5]–[7], [24]. These studies demonstrate the practical value of DRL for DFJSP: once trained, a policy can make rapid online decisions and adapt to stochastic state transitions without solving a new combinatorial optimization problem after every event. However, most reported gains are obtained at fixed training scales, under a single source of disturbance, or in specific dynamic settings. Practical deployment requires policies to transfer beyond the training scale and remain robust across disturbance regimes, a joint setting that remains less systematically evaluated in DFJSP [2], [8].

This setting is challenging for two reasons. First, scale expansion enlarges the state-action space and intensifies precedence-constrained resource competition, so dense operation-machine representations can suffer from rapidly increasing interaction cost and degraded structural transfer [9]. Second, shifts in disturbance regimes reshape both feasible actions and dispatching preferences. Job insertions increase the importance of release times and remaining workloads, machine failures affect availability and load balancing, and transportation-time variations alter the effective start times of subsequent operations. A monolithic policy trained across such regimes may blend incompatible decision patterns, causing policy interference under mixed conditions [10]–[12]. These challenges call for a scheduler that preserves precedence and resource-competition semantics with scale-efficient representations while conditioning its decisions on the observed disturbance context.

To address this underexplored setting of cross-scale transfer

Hongyao Ma, Jing Bi, Jiahui Zhai, and Junqi Zhang are with the College of Computer Science, Beijing University of Technology, Beijing, China (e-mails: hongyao-ma@emails.bjut.edu.cn, bijing@bjut.edu.cn, zhajiahui@emails.bjut.edu.cn). Ziqi Wang is with the School of Software Technology, Zhejiang University, Hangzhou, China (e-mail: wangz-qi0312@zju.edu.cn). Haitao Yuan is with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China (e-mail: yuan@buaa.edu.cn). MengChu Zhou is with the Faculty of Innovation Engineering, Macau University of Science and Technology, Avenida Wai Long, Taipa, Macau (e-mail: zhou@njit.edu). Jing Bi and Haitao Yuan are co-corresponding authors.

This work was supported by the National Key Research and Development Program of China under Grant 2025YFE0203700, the Fundo para o Desenvolvimento das Ciencias e da Tecnologia under Grant 0095/2025/AMJ, the Beijing Natural Science Foundation under Grant L233005, and the National Natural Science Foundation of China under Grant 62473014.

and robustness across disturbance regimes, this work proposes an **Event-Consistent Dual-Graph Mixture-of-Experts (E2G-MoE)** framework for DFJSP under concurrent disturbances. The main contributions are summarized as follows.

- 1) **Event-consistent concurrent disturbance environment.** A continuous-time event-driven DFJSP environment synchronizes dispatching with job insertions and machine failures and recoveries while incorporating a scenario-dependent and rollout-fixed transfer-time matrix within the same online decision interface.
- 2) **Scalable dual-graph representation.** A dual-graph encoder separates stable operation precedence from dynamic machine competition, enabling incremental operation expansion and sparse resource-competition updates.
- 3) **Disturbance-aware hierarchical policy for cross-regime robustness.** A hierarchical Mixture-of-Experts policy routes decisions by regime-level disturbance signals, reducing policy interference without disturbance-specific retraining.

II. RELATED WORK

A. Solution Approaches for DFJSP

DFJSP solution methods balance scheduling quality and responsiveness [19]. Exact optimization can be effective on small instances but scales poorly with problem size and disturbance frequency [20]. Heuristics are fast but rule-dependent, whereas meta-heuristics improve solution quality at higher computational cost [21]. Beyond conventional job-shop settings, machine-level collaborative scheduling has also been studied in heterogeneous multi-plant systems, where production, transportation, storage, substitution, and order-fulfillment constraints are jointly optimized using mathematical and meta-heuristic frameworks [18]. These studies highlight the importance of machine-level scheduling under heterogeneous industrial resources, but they mainly address collaborative planning and cost optimization rather than event-driven online DFJSP dispatching.

Event-driven predictive-reactive methods improve responsiveness by rescheduling at state-changing events and better match online shop-floor evolution than fixed-step rescheduling. However, their search or rule components must still be repeatedly invoked online, making response quality sensitive to the computation time available after each event. Recent work therefore increasingly uses reinforcement learning and data-driven strategies to amortize scheduling decisions across stochastic scenarios [17]. Once trained, these policies can incur much lower inference costs than iterative search, but their encoders must still capture scheduling dependencies effectively to generalize beyond the training-instance distribution.

B. Learning-Based Scheduling and Generalization

Learning-based scheduling aims to reduce reliance on manually designed dispatching rules and learn state-dependent decisions for dynamic manufacturing systems [22]. In terms of action-space design, existing DRL-based schedulers generally follow two paradigms: rule selection, in which the

policy chooses among predefined dispatching rules, and direct dispatching, in which the policy directly selects a feasible scheduling action [23]. Table I compares representative studies by problem type, objective, learning paradigm, and disturbance setting. Rule-selection methods inherit the interpretability and responsiveness of dispatching heuristics, but their action space is constrained by the predefined rule set, limiting their ability to represent fine-grained job-machine assignment preferences. Direct dispatching can learn richer state-dependent assignments over feasible job-machine pairs, yet it also exposes the learner to variable-size action sets, dynamic eligibility changes, and scale-dependent resource interactions. Its effectiveness therefore depends on whether the representation captures precedence, eligibility, resource load, and dynamic feasibility in a scalable form.

The generalization evidence summarized in Table I remains fragmented. Among the nine groups of prior studies, three primarily consider static or disturbance-free settings, two focus on dynamic job arrivals, one addresses transportation constraints, one adopts event-driven predictive-reactive scheduling, and two investigate broader dynamic or multi-agent settings. Collectively, these studies provide partial evidence across scale transfer, disturbance-regime transfer, transportation-aware scheduling, and distributed coordination. Existing evaluations typically examine these dimensions separately rather than within a unified setting. It remains necessary to investigate whether a policy trained on small-scale instances can transfer to larger instances and remain robust across varying concurrent-disturbance regimes without disturbance-specific retraining.

Recent DRL schedulers based on graph neural networks, attention mechanisms, and Transformer architectures have enhanced the representation of precedence constraints and resource-related information in JSP and FJSP by encoding operation relations and machine eligibility while capturing long-range dependencies [5], [13], [14]. Hierarchical DRL further decomposes scheduling decisions to accommodate large-scale and dynamic scheduling scenarios [6], [24]. MARL models jobs or machines as decentralized agents to support distributed coordination [25]–[28]. Reported results suggest that learned schedulers can generalize beyond the specific instances used during training. However, existing evaluations typically focus on individual aspects such as static structural generalization, dynamic job arrivals, transportation constraints, or distributed coordination. A policy for DFJSP under concurrent disturbances must jointly address structural distribution shifts induced by scale expansion and disturbance-regime shifts induced by changing disturbance combinations. Conditional policy architectures such as Mixture-of-Experts provide a structured mechanism for parameter sharing while allowing different experts to capture context-dependent decision preferences [10]–[12]. However, the joint evaluation of cross-scale transfer and robustness across concurrent-disturbance regimes in large-scale DFJSP remains underexplored.

Motivated by this gap, this work integrates an event-consistent decision interface, scalable dual-graph encoding, and disturbance-conditioned policy routing to support cross-scale transfer and robustness across disturbance regimes.

TABLE I
COMPARISON OF DRL-BASED JSP AND FJSP STUDIES.

Type	Method	Problem	Objective	Algorithm	Disturbance
<i>Single-Objective</i>					
Rule-Selection DRL	[4]	DFJSP	Single-Objective	Rule / DQN	Arrival
Direct-Dispatching DRL	[23]	JSP	Single-Objective	PPO	None
Graph-Based DRL	[5] / [7]	FJSP / DFJSP	Single-Objective	GNN / PPO	None / Transport
Attention-Based DRL	[13]	FJSP	Single-Objective	Attention / PPO	None
Hierarchical DRL	[24] / [6] / [16]	DFJSP / DDJSP / FJSP	Single-Objective	HRL / Multi-Action DRL	Arrival
MARL	[26] / [27] / [28]	DJSP / DJSP / DFJSP	Single-Objective	MARL	Dynamic / Dynamic / Dynamic
<i>Multiple-Objective</i>					
Rule-Selection DRL	[17] / [21]	DFJSP / DJSP	Multiple-Objective	Rule / DQN	Multi / Dynamic
Event-Driven DRL	[19]	DFJSP	Multiple-Objective	Predictive-Reactive DRL	Event
End-to-End DRL	[15]	FJSP	Multiple-Objective	End-to-End DRL	None
Ours	E2G-MoE	DFJSP	Single-Objective	GNN / PPO + HMoE	Multi

III. PROBLEM FORMULATION

A. FJSP Preliminaries

A flexible job shop consists of a set of jobs $\mathcal{J} = \{1, \dots, |\mathcal{J}|\}$ and a set of machines $\mathcal{M} = \{1, \dots, |\mathcal{M}|\}$. Each job $j \in \mathcal{J}$ comprises an ordered sequence of operations $\{O_{j,1}, \dots, O_{j,K_j}\}$ subject to technological precedence constraints. Operation $O_{j,k}$ can be processed on an eligible machine $m \in \mathcal{M}_{j,k} \subseteq \mathcal{M}$ with processing time $p_{j,k}^m > 0$. A schedule assigns each operation to an eligible machine and determines its start and completion times $(S_{j,k}, C_{j,k})$. Under uninterrupted processing, $C_{j,k} = S_{j,k} + p_{j,k}^m$.

B. DFJSP Formulation

1) *Dynamic Flexible Job Shop Scheduling*: Each job j has a release time r_j , and production may experience job insertions, machine failures and recoveries, and scenario-dependent transportation times. DFJSP is formulated as an event-driven dispatching process with decisions made at event-triggered decision epochs. Its scheduling structure involves (i) stable technological precedence within jobs and (ii) resource feasibility and competition induced by machine availability, routing flexibility, and disturbances.

2) *Two-Layer Disjunctive Representation*: DFJSP is represented by a two-layer disjunctive graph, consisting of an operation precedence layer and a machine transportation layer (Fig. 1). Operations are nodes linked by directed technological precedence edges, which remain unchanged after job release and are extended only for inserted jobs. Machines are nodes connected by transportation relations represented by a delay matrix $\mathbf{T} \in \mathbb{R}_{\geq 0}^{|\mathcal{M}| \times |\mathcal{M}|}$ and transfer time $T_{m' \rightarrow m}$. Cross-layer links connect each operation to its eligible machines $\{\mathcal{M}_{j,k}\}$. Disturbances alter machine availability and feasibility, but the underlying disjunctive structure remains unchanged, motivating the dual-graph representation in Section IV.

C. Event-Triggered Dispatching Feasibility and Objective

Under the event-driven continuous-time formulation introduced above, scheduling in DFJSP is modeled as an online dispatching process. Decisions are made exclusively at event-triggered decision epochs. Let $\mathcal{E}$ denote an event set, including job releases and insertions, operation completions, machine failures and recoveries, and machine idleness. Scheduling decisions are permitted only at the sequence of event-triggered decision epochs $\{t_k\}_{k \geq 0}$.

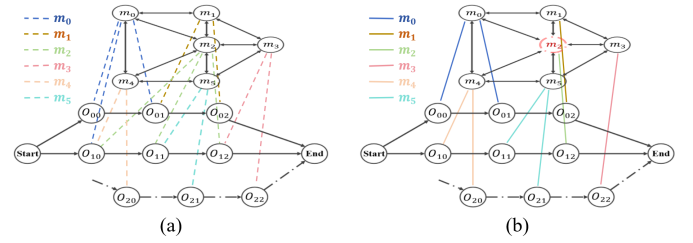


Fig. 1. Two-layer disjunctive representation of DFJSP.

At each decision epoch t_k , the system state $\mathcal{S}(t_k)$ is updated based on the events that have occurred, and the scheduler selects a feasible job-machine assignment. Between two consecutive decision epochs, the system evolves autonomously following the underlying production dynamics.

For each job j , let $\kappa(j, t)$ denote the index of its next operation at time t . If $\kappa(j, t) > 1$, $m'(j, t)$ denotes the machine that processes the predecessor operation $O_{j, \kappa(j, t)-1}$, which is completed at $C_{j, \kappa(j, t)-1}$; otherwise, the predecessor-completion term is omitted, equivalently by setting $C_{j,0} = r_j$ and $T_{\emptyset \rightarrow m} = 0$. Let $a_m(t)$ denote the earliest availability time of machine m , accounting for its workload and failure-induced downtime. The earliest feasible start time for assigning the current operation of job j to machine m is given as:

$$\text{pft}_{j,m}(\mathcal{S}(t)) = \max \left\{ r_j, a_m(t), C_{j, \kappa(j, t)-1} + T_{m'(j, t) \rightarrow m} \right\}. \quad (1)$$

At decision epoch t_k , a job-machine pair (j, m) is included in the dispatching candidate set if $m \in \mathcal{M}_{j, \kappa(j, t_k)}$ and $\text{pft}_{j,m}(\mathcal{S}(t_k)) \leq t_k$.

All such dispatchable pairs constitute the admissible action set at t_k . Accordingly, this is a non-delay policy class. Whenever a feasible pair exists, the policy assigns it rather than deliberately idling, and no wait action is included. The definition implicitly enforces the following feasibility constraints.

- 1) **Release feasibility and dynamic arrivals**: For each job j , the first operation satisfies $S_{j,1} \geq r_j$. Dynamically inserted jobs become schedulable only after their release.
- 2) **Machine compatibility**: If operation $O_{j,k}$ is processed on machine m , $m \in \mathcal{M}_{j,k}$.
- 3) **Machine capacity**: Each machine can process at most

one operation at any time, and processing intervals assigned to the same machine cannot overlap.

- 4) **Machine availability under failures:** Let D_m denote the union of the downtime intervals of machine m . An operation assigned to m is suspended during downtime and resumes on the same machine after recovery, such that $\mu([S_{j,k}, C_{j,k}) \setminus D_m) = p_{j,k}^m$, where $\mu(\cdot)$ denotes the duration of a time interval.
- 5) **Precedence and transport:** For each job j and $k \geq 2$, if $O_{j,k-1}$ and $O_{j,k}$ are assigned to machines m' and m , respectively, then $S_{j,k} \geq C_{j,k-1} + T_{m' \rightarrow m}$.

IV. PROPOSED METHOD

E2G-MoE formulates DFJSP under concurrent disturbances as an event-triggered online scheduling problem. At each decision epoch, the environment constructs a dispatching candidate set of job-machine pairs, the dual-graph encoder extracts precedence and machine-competition representations, and the disturbance-aware HMoE actor scores the candidate pairs and selects an assignment. The policy is trained with PPO using the event-driven simulator described below.

A. Event-Consistent Environment

DFJSP evolves through discrete shop-floor events, including job arrivals, operation completions, and machine failures and recoveries. Transportation times are instead modeled as scenario-dependent quantities that remain fixed within each realized rollout. The environment models continuous-time dynamics and invokes the agent only at event-triggered decision epochs. A physical event time may induce multiple sequential dispatching decisions without advancing the clock. As illustrated in Fig. 2, the environment provides a structured observation and a dynamic feasibility mask at each decision epoch.

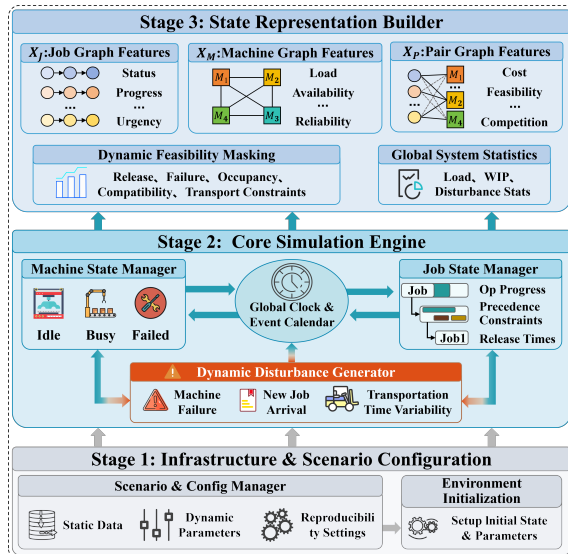


Fig. 2. Event-consistent environment and decision interface.

1) **Event-Triggered Decision Epochs:** Let k denote the decision index and t_k its physical time stamp, where $t_{k+1} \geq t_k$. At each new event time, disturbances observed up to t_k are processed first. A job-insertion event releases a pre-sampled job and appends its operation subgraph to the current instance. Transportation variability is represented by a seed-controlled and scenario-level matrix T generated before rollout and held fixed throughout that rollout. For a candidate machine m , the transfer time from the job's previous machine enters its earliest feasible start time and the dispatching mask. T is not updated by a transportation-change event. A machine failure renders the affected machine unavailable and suspends any operation in progress. Upon recovery, the interrupted operation resumes on the same machine until its remaining processing requirement is completed. The resulting state is then used to reconstruct the graph observation, feasibility mask, and candidate job-machine pairs. When multiple assignments are feasible at the same physical time, the policy selects one job-machine pair from the complete current admissible set and factorizes concurrent allocation autoregressively rather than following a predetermined job or machine order. After each selected pair is committed, the state, graph observation, candidate set, and feasibility mask are updated, and the remaining pairs are re-scored. Later decisions depend on earlier same-time commitments while physical time remains unchanged ($t_{k+1} = t_k$). The simulation clock advances to the next event time once no currently dispatchable pair remains. Thus, each reinforcement learning step corresponds to one assignment decision under a consistent event-triggered decision interface.

This interface induces a finite-horizon online MDP $\langle \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle$ at event-triggered decision epochs, where $a_k = (j, m)$ is restricted to $\mathcal{A}(s_k) = \{(j, m) \mid M_k(j, m) = 1\}$. If $\mathcal{A}(s_k) = \emptyset$, the simulation clock advances without a policy action. When feasible assignments exist, the agent selects a job-machine pair from the masked action set.

2) Structured Observation and Dynamic Feasibility Mask:

At decision epoch k , the environment provides a structured observation s_k representing the current decision state S_k at physical time t_k . The observation is defined as:

$$s_k = \{X_k^{(\mathcal{J})}, X_k^{(\mathcal{M})}, X_k^{(\mathcal{P})}, \phi_k, M_k\}. \quad (2)$$

where $X_k^{(\mathcal{J})}$, $X_k^{(\mathcal{M})}$, and $X_k^{(\mathcal{P})}$ denote operation-level, machine-level, and candidate job-machine pair features, respectively. M_k is the dispatching mask for currently feasible job-machine pairs. ϕ_k is a six-dimensional disturbance descriptor constructed from information available no later than t_k . Its six entries are, in order, keep (i) the number of jobs not yet released, (ii) the number of job releases observed during the preceding 10 physical-time units, (iii) the mean positive inter-machine transportation time, (iv) the maximum positive inter-machine transportation time, (v) the number of machines currently unavailable because of failure, and (vi) the number of failure-onset events observed during the preceding 10 physical-time units. The transportation statistics are computed over the positive entries of the rollout-fixed matrix T . The complete job set $\mathcal{J}$ is assumed known, whereas the individual release times of unreleased jobs are hidden from

the policy. All six entries are unnormalized raw counts or physical-time values. If a source set is empty, its corresponding statistic is set to zero. These components are further encoded by the scalable graph-structured state representation described in Section IV-B.

To construct the dispatching candidate set, a binary mask M_k is defined over candidate job-machine pairs:

$$M_k(j, m) = \begin{cases} 1, & m \in \mathcal{M}_{j, \kappa(j, t_k)} \text{ and } \text{pft}_{j, m}(\mathcal{S}_k) \leq t_k, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

where $\text{pft}_{j, m}(\mathcal{S}_k)$ denotes the earliest feasible start time of the current operation of job j on machine m , jointly considering job release, precedence completion, transportation time, machine availability, failure-induced downtime, and assignments already made at t_k .

B. Scalable Graph-Structured State Representation

As illustrated in Fig. 3, the proposed Scalable Graph-Structured State Representation (SGSR) captures event-driven DFJSP states by decomposing them into stable technological precedence and time-varying resource competition. Based on the two-layer disjunctive formulation, SGSR constructs two sparse graphs:

- **Precedence-Constrained Operation Graph:** A structurally stable operation-level graph that encodes technological dependencies and accommodates job insertions through incremental subgraph expansion without modifying existing connections.
- **Competition-Aware Machine Graph:** A machine-level relational graph over fixed machine nodes, whose neighborhoods are reconstructed at each decision epoch from the current feasible assignment set.

The operation graph captures the relatively stable technological precedence structure, whereas the machine graph is reconstructed from the current feasible set to capture time-varying resource contention. Candidate-level fusion combines job-progress information with current machine competition during action scoring while preserving sparse cross-entity propagation.

1) *Precedence-Constrained Operation Encoding:* The operation encoder models precedence relations as structurally stable dependencies within each job. New operation subgraphs are appended for dynamically inserted jobs without modifying existing graph connections. To avoid dense message passing, each operation aggregates information within a fixed-size local technological neighborhood consisting of predecessor (pred), self, and successor (succ) relations. Relations that violate boundary validity or event-consistent visibility are masked before attention normalization.

For an operation node i , let $\mathcal{R} = \{\text{pred}, \text{self}, \text{succ}\}$ denote the relation set, and let $r(i)$ represent the neighbor of i under relation $r \in \mathcal{R}$. At each decision epoch t_k , a binary visibility indicator $\delta_{i, r}(t_k) \in \{0, 1\}$ enforces boundary validity and event-consistent visibility while preserving the graph structure. The corresponding precedence-aware aggregation at layer l is

formulated as:

$$\mathbf{h}_i^{(l+1)} = \sum_{r \in \mathcal{R}} \alpha_{i, r}^{(l)} W_V \mathbf{h}_{r(i)}^{(l)}, \quad (4)$$

where the masked attention weight is computed as:

$$\alpha_{i, r}^{(l)} = \frac{\exp(e_{i, r}^{(l)} \delta_{i, r}(t_k))}{\sum_{r' \in \mathcal{R}} \exp(e_{i, r'}^{(l)} \delta_{i, r'}(t_k))}, \quad (5)$$

and the unnormalized attention score is given by:

$$e_{i, r}^{(l)} = \mathbf{a}_{\mathcal{J}}^\top [W_Q \mathbf{h}_i^{(l)} \parallel W_K \mathbf{h}_{r(i)}^{(l)}] + g(r), \quad (6)$$

where $g(r)$ is a learnable relation-specific bias for $r \in \mathcal{R}$, encoding the technological role of each precedence direction.

At each epoch, only the current operation of each job is dispatchable. Completion status, remaining workload, and waiting duration are reflected in node features and visibility masks. Since each operation attends to a constant-size neighborhood, the per-layer aggregation cost is $\mathcal{O}(N_{\text{op}})$.

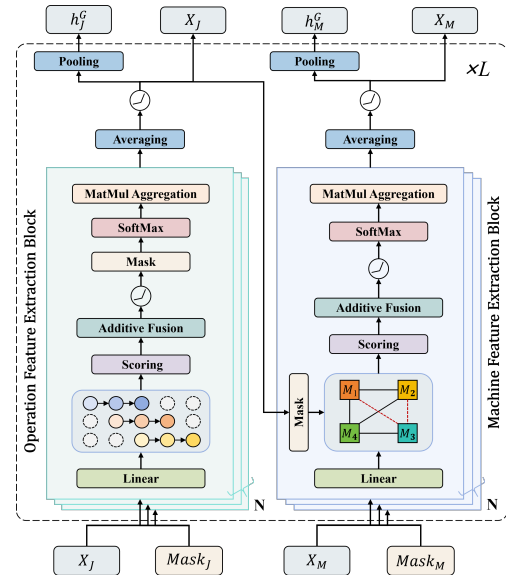


Fig. 3. Dual-graph encoding of precedence and machine competition.

2) *Competition-Aware Machine Encoding:* The machine layer captures feasibility-induced, time-varying resource interactions over the fixed physical machine set $\mathcal{M}$.

Let $\mathcal{F}_k$ denote the feasible job-machine assignment set at decision epoch t_k . For each machine $m \in \mathcal{M}$, the associated candidate set of jobs is defined as:

$$\mathcal{S}_k(m) = \{j \in \mathcal{J} \mid (j, m) \in \mathcal{F}_k\}. \quad (7)$$

Machine interactions are induced by overlaps between candidate sets of jobs, restricting message passing to competition-relevant machine pairs.

Specifically, for machines m and m' , the competition context is defined as:

$$\mathbf{c}_{m, m'}^{(l)} = \sum_{j \in \mathcal{S}_k(m) \cap \mathcal{S}_k(m')} \mathbf{h}_j^{\text{cand}, (l)}, \quad (8)$$

where $\mathbf{h}_j^{\text{cand}, (l)}$ is the layer- l embedding of job j 's current

TABLE II
MESSAGE-PASSING EDGE REDUCTION.

Scale	Nodes	Single [5]	SGSR	Reduction	Ratio
10 × 5	55	435	155	64.37%	2.81
30 × 10	310	4,150	940	77.35%	4.41
50 × 10	510	6,910	1,500	78.29%	4.61
100 × 25	2,525	72,325	7,925	89.04%	9.13

operation. A nonempty intersection indicates competition for at least one common candidate job.

Let $\mathbf{g}_m^{(l)}$ denote the layer- l embedding of machine m , encoding its dynamic state such as availability and workload. Machine representations are updated over a sparse competition neighborhood:

$$\mathbf{g}_m^{(l+1)} = \sum_{m' \in \mathcal{N}_M(m)} \beta_{m,m'}^{(l)} W_V \mathbf{g}_{m'}^{(l)}, \quad (9)$$

where $\mathcal{N}_M(m)$ is induced by $\mathcal{S}_k(m) \cap \mathcal{S}_k(m') \neq \emptyset$. The corresponding attention coefficient is:

$$\beta_{m,m'}^{(l)} = \text{softmax}_{m' \in \mathcal{N}_M(m)} \left(e_{m,m'}^{(l)} \right), \quad (10)$$

where the score is defined as:

$$e_{m,m'}^{(l)} = \mathbf{a}_M^\top \left[W_Q \mathbf{g}_m^{(l)} \parallel W_K \mathbf{g}_{m'}^{(l)} \right] + \rho \left(\mathbf{c}_{m,m'}^{(l)} \right). \quad (11)$$

The first term captures state-conditioned machine relevance, while the second captures contention among machines that share feasible candidates. Thus, machine interactions are re-configured by the current competition structure without dense job-machine enumeration. To quantify this structural reduction, Table II reports the edge-level message-passing statistics, where Single denotes the unified operation-machine graph constructed following the graph-based DRL implementation of Song *et al.* [5]. SGSR keeps the same scheduling entities and feasible job-machine action space as this single-graph reference, but reduces redundant propagation by separating local precedence aggregation from competition-aware machine updating. The propagated edges are reduced from 435 to 155 on 10×5 and from 72,325 to 7,925 on 100×25 , demonstrating a substantially more compact message-passing structure as the problem size grows. Operation-machine interactions are retained at the candidate-pair fusion and masked action-scoring stage.

C. Hierarchical Mixture-of-Experts Actor

Concurrent releases, failures, and transportation delays make dispatching preferences depend on the disturbance regime. A monolithic policy may blend incompatible patterns across regimes, causing policy interference. As illustrated in Fig. 4, the HMoE actor uses global disturbance context to route candidate-pair representations among experts, while shared projections help stabilize training.

1) *Global-Context MoE Routing*: At decision epoch t_k , SGSR produces operation and machine embeddings, which are aligned with candidate job-machine pair features and projected into a shared hidden space. The routing input concatenates mask-aware global summaries of the valid operation

and machine embeddings with the six-dimensional disturbance descriptor ϕ_k defined in Section IV-A.2.

Let $\mathcal{P}_k = \{(j_q, m_q)\}_{q=1}^{|\mathcal{P}_k|}$ denote the ordered set of candidate job-machine pairs at decision epoch t_k . Accordingly, $\tilde{H}_k^{(O)}$ and $\tilde{H}_k^{(M)}$ denote the pair-aligned operation and machine embedding matrices, respectively, obtained by gathering the current operation embedding of job j_q and the embedding of machine m_q for each $(j_q, m_q) \in \mathcal{P}_k$. The initial pair-level representation is then defined as:

$$H_k^{(0)} = f_{\text{pre}} \left(\tilde{H}_k^{(O)} \parallel X_k^{(P)} \parallel \tilde{H}_k^{(M)} \right) \in \mathbb{R}^{|\mathcal{P}_k| \times d_h}, \quad (12)$$

where $\parallel$ denotes feature-wise concatenation and f_{pre} is applied row-wise, mapping operation, pair, and machine features into a common latent space.

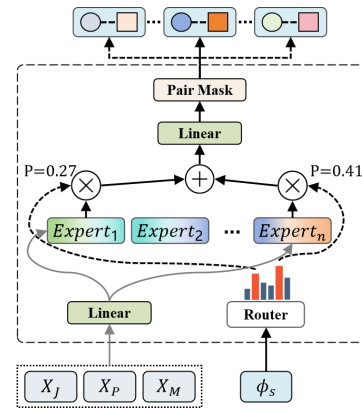


Fig. 4. Disturbance-conditioned hierarchical MoE actor.

To enable regime-aware specialization, these inputs form the global routing context $\mathbf{c}_k$. The router produces a sparse distribution over E experts:

$$\omega_k = \text{softmax}(\text{TopK}(G(\mathbf{c}_k) + \epsilon_k, K_r)), \quad (13)$$

where $\mathbf{c}_k$ concatenates the pooled operation representation, pooled machine representation, and the six-dimensional disturbance descriptor ϕ_k in Section IV-A.2; $G(\cdot)$ is a two-layer MLP with 64 hidden units and ReLU activation; $\text{TopK}(\cdot, K_r)$ retains the K_r largest routing logits before normalization; and ϵ_k is zero-mean Gaussian gating noise used only during training. We use $E = 8$ and $K_r = 2$.

Each selected expert $f_e(\cdot)$ models regime-specific dependencies among candidate job-machine pairs in the shared hidden space and produces an expert-level representation. The selected expert outputs are fused according to ω_k and mapped to unnormalized action logits through a shared post-projection:

$$z_k = f_{\text{post}} \left(\sum_{e: \omega_{k,e} > 0} \omega_{k,e} f_e(H_k^{(0)}) \right). \quad (14)$$

The resulting logits are masked and normalized over feasible job-machine pairs based on the feasibility mask in Section IV-A.2 to ensure constraint-consistent action selection. Here, $f_e(\cdot)$ denotes expert e , $\omega_{k,e}$ is its routing weight, and f_{post} is the shared post-projection producing the unnormalized action logits z_k . Since different experts transform the same candidate-

pair representation differently, changes in ω_k alter the resulting expert mixture and hence the relative preferences among feasible job–machine actions.

2) Optimization Objective: The HMoE actor is trained within an actor–critic framework based on proximal policy optimization (PPO). A load-balancing regularizer is incorporated into the routing distribution to prevent expert collapse and promote functional specialization. The overall objective follows the standard PPO formulation augmented with this auxiliary regularization term, as described in Section IV-D.

Algorithm 1 Training procedure of E2G-MoE.

```

1: Input: Training and validation pools  $\mathcal{D}_{\text{tr}}$  and  $\mathcal{D}_{\text{val}}$ ; seed sets  $\mathcal{Z}_{\text{tr}}$  and  $\mathcal{Z}_{\text{val}}$ ;
   activated disturbance set  $\mathcal{U}_{\text{act}}$ 
2: Initialize: Actor  $\pi_\theta$ , critic  $V_\psi$ , rollout buffer  $\mathcal{B}$ , and  $N_{\text{env}}$  parallel environments
3:  $\tilde{\mathcal{D}}_{\text{tr}} \leftarrow \text{Aug}(\mathcal{D}_{\text{tr}}, \mathcal{Z}_{\text{tr}}, \mathcal{U}_{\text{act}})$ 
4:  $\tilde{\mathcal{D}}_{\text{val}} \leftarrow \text{Aug}(\mathcal{D}_{\text{val}}, \mathcal{Z}_{\text{val}}, \mathcal{U}_{\text{act}})$ 
5: for  $n \leftarrow 1$  to  $N_{\text{upd}}$  do
6:   Reset  $\mathcal{B}$  and sample training scenarios from  $\tilde{\mathcal{D}}_{\text{tr}}$ 
7:   while  $|\mathcal{B}| < N_{\text{roll}}$  do
8:     for each parallel environment  $v$  do
9:       Construct  $\mathbf{M}_k^v$  and obtain  $(\mathbf{z}_k^v, \omega_k^v) \leftarrow \pi_\theta(s_k^v)$ 
10:       $\pi_k^v \leftarrow \text{MaskedSoftmax}(\mathbf{z}_k^v, \mathbf{M}_k^v)$ 
11:      Sample  $a_k^v \sim \pi_k^v$  and observe  $(r_k^v, s_{k+1}^v, d_k^v)$ 
12:      Store the transition and log  $\pi_k^v(a_k^v)$  in  $\mathcal{B}$ 
13:      if  $d_k^v = 1$  then
14:        Reset environment  $v$  with a sampled scenario
15:      end if
16:    end for
17:  end while
18:  Compute  $\hat{R}_k$  and  $\hat{A}_k$  from  $\mathcal{B}$ 
19:  for  $e \leftarrow 1$  to  $K_{\text{PPO}}$  do
20:    for each minibatch  $\mathcal{B}_i \subset \mathcal{B}$  do
21:      Compute  $\mathcal{L}_{\text{PPO}}$  and  $\mathcal{L}_{\text{bal}}$  on  $\mathcal{B}_i$ 
22:       $(\theta, \psi) \leftarrow (\theta, \psi) - \alpha \nabla_{\theta, \psi} [\mathcal{L}_{\text{PPO}} + \lambda_{\text{bal}} \mathcal{L}_{\text{bal}}]$ 
23:    end for
24:  end for
25:  if  $n \bmod N_{\text{aug}} = 0$  then
26:    Regenerate  $\mathcal{D}_{\text{tr}}$  using  $\text{Aug}(\cdot)$ 
27:  end if
28:  if  $n \bmod N_{\text{val}} = 0$  then
29:    Evaluate  $\pi_\theta$  on  $\tilde{\mathcal{D}}_{\text{val}}$ 
30:  end if
31: end for

```

D. Training and Reward Design

The HMoE actor is trained with an on-policy PPO actor–critic framework, as summarized in Algorithm 1. Dynamic training and validation scenarios augment static 10×5 DFJSP pools with seed-controlled job-insertion, machine-failure, and scenario-dependent transportation-time realizations. A single policy is trained with the job-insertion and machine-failure operators active together with scenario-level transportation variability, and is evaluated under S , I , $I + F$, and $I + F + T$ settings without disturbance-specific retraining.

Within each PPO minibatch, a load-balancing regularizer discourages expert collapse by penalizing deviations of the batch-averaged expert usage from a uniform distribution together with excessive usage variance across experts. The resulting objective is:

$$\mathcal{L}(\theta, \psi) = \mathcal{L}_{\text{PPO}}(\theta, \psi) + \lambda_{\text{bal}} \mathcal{L}_{\text{bal}}, \quad (15)$$

where $\mathcal{L}_{\text{PPO}}$ is the standard PPO loss, $\mathcal{L}_{\text{bal}}$ is the load-balancing regularizer described above, and λ_{bal} controls its contribution. To mitigate sparse terminal rewards, the environment maintains a normalized completion lower bound. *Tilde*s

denote division by a positive, policy-independent time scale b fixed throughout each rollout, i.e., $\tilde{x} = x/b$. The bound is initialized from the cumulative minimum eligible processing times of each job. After assigning operation O_{j, ℓ_a} to machine m at epoch k , the causal increase of its normalized completion lower bound is defined as:

$$\Delta_k = \left[\tilde{t}_k + \tilde{p}_{j, \ell_a}^m - \widetilde{LB}_{j, \ell_a}^{(k)} \right]^+, \quad [x]^+ = \max\{x, 0\}. \quad (16)$$

This calculation uses only information available when the assignment is committed, and unobserved future failures affect later states only after their events are observed. The downstream update is defined as:

$$\widetilde{LB}_{i, \ell}^{(k+1)} = \begin{cases} \widetilde{LB}_{i, \ell}^{(k)} + \Delta_k, & i = j, \ell \geq \ell_a, \\ \widetilde{LB}_{i, \ell}^{(k)}, & \text{otherwise.} \end{cases} \quad (17)$$

Hence, only the downstream entries of the selected job are modified. The update has linear complexity in the number of operations of job j , i.e., $\mathcal{O}(K_j)$. The lower-bound entries for newly inserted jobs are added to the maintained array at their release events.

The potential and immediate reward are defined as:

$$\Phi(s_k) = \max_{j, \ell} \widetilde{LB}_{j, \ell}^{(k)}, \quad r_k = \Phi(s_k) - \Phi(s_{k+1}). \quad (18)$$

where $\Phi(s_k)$ is the maximum maintained normalized completion lower bound at state s_k . When $\gamma = 1$, the return telescopes to $\Phi(s_0) - \Phi(s_T)$. Thus, for a fixed realized scenario, maximizing the return minimizes the terminal maintained bound. Because this causal bound need not equal the realized makespan before all disturbance effects are incorporated, the reward is used as a dense and makespan-oriented surrogate rather than as an unconditional equivalent of the physical-time makespan.

V. EXPERIMENTS

This section evaluates E2G-MoE along five dimensions: disturbance-free competitiveness, transfer beyond the training scale and benchmark family, robustness to compounding disturbances, the contribution of each architectural component, and regime-dependent expert routing.

A. Experimental Settings

TABLE III
DATASET STATISTICS AND REALIZED DISTURBANCE INTENSITIES.

Dataset	$\bar{N}_{\text{op}}$	$\bar{f}/ \mathcal{M} $	$\eta_{\text{ins}}/\phi_{\text{fail}}/CV_{\text{trans}}$
10×5	50	0.576	0.167/0.012/0.364
30×10	300	0.605	0.167/0.012/0.397
50×10	500	0.553	0.167/0.011/0.419
100×25	2,500	0.521	0.167/0.012/0.397
Fattahi	17.40	0.517	0.198/0.067/0.326
Barnes	158.33	0.089	0.167/0.023/0.420
Dauzere	292.00	0.330	0.167/0.006/0.407
Brandimarte	141.40	0.356	0.167/0.263/0.400

TABLE IV
STATIC SCHEDULING PERFORMANCE ON SYNTHETIC FJSP INSTANCES.

Dataset	Metric	CRJ.EET	CRJ.EETLQ	CRJ.EETD	GA	CP-SAT	DRL	DANIEL	MAPPO	MPGN	Ours
10×5	Makespan	352.00	483.00	483.00	345.00	236.00	499.00	342.00	391.00	356.00	326.00
	Gap (%)	49.15	104.66	104.66	46.19	0.00	111.44	44.92	65.68	50.85	38.14
	Time (s)	0.013	0.023	0.020	10.32	51.69	0.40	0.38	1.14	0.45	0.40
30×10	Makespan	864.00	934.00	877.00	885.00	650.00	1334.00	812.00	936.00	827.00	714.00
	Gap (%)	32.92	43.69	34.92	36.15	0.00	105.23	24.92	44.00	27.23	9.85
	Time (s)	0.063	0.10	0.11	26.20	120.19	1.55	2.01	197.36	1.83	1.22
50×10	Makespan	1278.90	1351.10	1387.80	2143.19	1629.10	4818.70	1295.30	1886.75	1255.80	1286.10
	Gap (%)	1.84	7.59	10.51	70.66	29.73	283.72	3.15	50.24	0.00	2.41
	Time (s)	0.23	0.24	0.23	47.24	600.16	4.58	4.24	566.82	8.23	4.21
100×25	Makespan	928.00	943.50	943.70	1964.16	1043.70	3485.40	893.70	1386.50	1388.05	887.30
	Gap (%)	4.59	6.33	6.36	121.36	17.63	292.81	0.72	56.26	56.44	0.00
	Time (s)	1.13	1.11	1.15	148.22	2400.00	89.98	85.69	9083.39	110.66	86.74

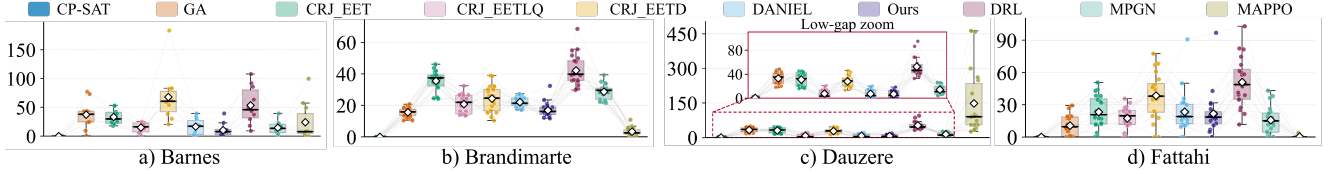


Fig. 5. Static performance distributions on the four public benchmark families.

1) *Datasets and Disturbance Generation*:¹ Experiments use synthetic DFJSP instances at 10×5 , 30×10 , 50×10 , and 100×25 scales and four standard FJSP benchmark sets, namely Barnes, Brandimarte, Dauzere, and Fattahi, for cross-dataset transfer. All learning-based methods are trained only on 10×5 instances. Larger synthetic instances test scale extrapolation, while public benchmarks test transfer to different routing-flexibility and eligibility distributions. $\bar{N}_{op}$ denotes the mean number of operations per instance before disturbance augmentation. The realized disturbance intensity is quantified by the inserted-operation ratio η_{ins} , normalized downtime exposure ϕ_{fail} , and transportation-time coefficient of variation CV_{trans} . The disturbance-free regime is denoted by S , while I , $I + F$, and $I + F + T$ progressively incorporate job insertion, machine failure/recovery, and transportation-time variability, respectively. Table III summarizes the corresponding dataset statistics and realized disturbance intensities.

For each static instance, a seed-controlled dynamic scenario is generated before the environment is reset. No additional disturbances are sampled during the rollout. In regime I , $K^{ins} \sim U_{\mathbb{Z}}(0, K_{max}^{ins})$ inserted jobs are appended, with $K_{max}^{ins} = 9$ by default; each inserted job has $U_{\mathbb{Z}}(3, 6)$ operations, uniformly sampled eligible machines, processing times from $U_{\mathbb{Z}}(5, 40)$, and release time from $U_{\mathbb{Z}}(10, 90)$. In regime $I + F$, each machine has $K_m^F \sim U_{\mathbb{Z}}(0, \min(3, F_{max}))$ downtime intervals, where $F_{max} = 5$, $s \sim U(10, 90)$, and $e \sim U(s + 1, 100)$. In regime $I + F + T$, transportation-time variability is represented by a scenario-level matrix T with $T_{ii} = 0$, $b_{ij} \sim U(5, 30)$, $\epsilon_{ij}, \epsilon_{ji} \sim U(-2, 2)$, $T_{ij} = \max(0, b_{ij} + \epsilon_{ij})$, and $T_{ji} = \max(0, b_{ij} + \epsilon_{ji})$. The same generated realizations are reused for all compared methods under the same random seeds.

2) *Baselines and Evaluation Metrics*: For static scheduling, we compare E2G-MoE with three dispatching rules, a genetic algorithm (GA), time-limited CP-SAT, and four learning-

based baselines: DRL [5], DANIEL [13], MAPPO [15], and MPGN [16]. The rule-based baselines are the three rules with the best average ranks in a feasible library of 60 compound rules. MAPPO serves as the representative MARL baseline. All four learning-based baselines use publicly released implementations from the corresponding authors. The dynamic comparison is restricted to learning-based online dispatchers under a common event-driven simulator, disturbance protocol, dispatching mask, training budget, test instances, and realized disturbance scenarios. Rolling-horizon and predictive-reactive methods use a separate online reoptimization protocol and are outside this comparison.

3) *Configurations*: All learning-based methods are trained on the same synthetic 10×5 pool, which is split into training, validation, and test subsets with an 8:1:1 ratio. For dynamic evaluation, all DRL baselines and E2G-MoE use the same seed-controlled concurrent-disturbance augmentation during training. They also share the event-driven simulator, disturbance protocol, dispatching mask, and training budget. Larger synthetic instances and public benchmarks are used only for zero-shot evaluation without fine-tuning. All reported results are averaged over five independent random seeds; all methods share the same test instances and realized disturbance scenarios.

For E2G-MoE, the operation and machine encoders each contain two four-head attention layers with input/output dimensions of 10/32 and 8/8, respectively. The actor and critic are three-layer MLPs with 64 hidden units. HMoE contains eight single-layer experts with 64-dimensional hidden states, Top-2 routing, training-only gate noise of 0.2, the routing context c_k defined in Section IV-C.1, and $\lambda_{bal} = 0.2$. PPO is trained for 1000 updates using five parallel environments, a learning rate of 3×10^{-4} , $\gamma = 1.0$, a GAE parameter of 0.98, a clipping parameter of 0.2, and a minibatch size of 2048. Each update performs four PPO epochs, with policy, value, and entropy coefficients of 1.0, 0.5, and 0.01, respectively.

¹Benchmark instances are available at <https://github.com/ScheduleOpt/benchmarks/tree/main/flexible-jobshop/instances/fjsp>.

B. Static Scheduling Performance

The static experiment evaluates disturbance-free FJSP dispatching performance across four problem scales. Table IV reports the mean makespan, relative gap, and runtime at each scale. CP-SAT uses time budgets of 60, 120, 600, and 2400 s for the 10×5 , 30×10 , 50×10 , and 100×25 instances, respectively. The reported gaps are computed with respect to the best mean makespan among the compared methods in each setting. E2G-MoE ranks first among learning-based methods on the 10×5 , 30×10 , and 100×25 instances and second on 50×10 . Although CP-SAT obtains better makespans on the two smaller scales, E2G-MoE obtains lower mean makespans than the CP-SAT incumbent solutions returned within the specified time limits on two larger scales, indicating competitive learning-based static performance across tested scales.

C. Cross-Dataset Transfer

TABLE V
DYNAMIC SCHEDULING PERFORMANCE ON SYNTHETIC DFJSP INSTANCES.

Dataset	Regime	Metric	DANIEL	DRL	MAPPO	MPGN	Ours
10×5	I	Makespan	333.80	663.80	392.50	342.70	329.30
		Gap (%)	1.37	101.58	19.19	4.07	0.00
		Time (s)	0.63	0.57	1.25	0.68	0.57
	I+F	Makespan	389.88	764.61	458.70	407.50	386.81
		Gap (%)	0.79	97.67	18.59	5.35	0.00
		Time (s)	0.52	0.56	1.28	0.70	0.62
	I+F+T	Makespan	404.54	777.19	480.60	425.80	396.42
		Gap (%)	2.05	96.05	21.24	7.41	0.00
		Time (s)	0.52	0.55	1.29	0.69	0.52
30×10	I	Makespan	772.00	1530.60	918.80	746.70	702.60
		Gap (%)	9.88	117.85	30.77	6.28	0.00
		Time (s)	1.70	2.22	216.02	1.90	1.75
	I+F	Makespan	851.54	1663.87	1035.60	816.59	761.87
		Gap (%)	11.76	118.39	35.93	7.17	0.00
		Time (s)	1.74	2.24	220.21	1.94	1.76
	I+F+T	Makespan	855.39	1619.79	1054.80	838.20	773.42
		Gap (%)	10.60	109.43	36.38	8.38	0.00
		Time (s)	1.74	2.27	222.55	1.91	1.73
50×10	I	Makespan	1394.90	5422.00	2289.34	1392.40	1349.80
		Gap (%)	3.34	301.69	69.61	3.16	0.00
		Time (s)	6.62	6.95	620.42	9.20	6.12
	I+F	Makespan	1538.21	5746.87	2598.76	1579.20	1457.40
		Gap (%)	5.54	294.32	78.31	8.36	0.00
		Time (s)	6.76	6.03	632.46	9.31	6.24
	I+F+T	Makespan	1601.86	5989.50	2762.93	1667.52	1506.70
		Gap (%)	6.31	297.52	83.38	10.67	0.00
		Time (s)	6.81	6.10	639.18	9.38	6.31
100×25	I	Makespan	947.30	3833.90	1608.30	1513.01	922.80
		Gap (%)	2.65	315.46	74.28	63.96	0.00
		Time (s)	124.60	123.80	10086.26	181.20	122.10
	I+F	Makespan	1018.40	4112.80	1788.60	1651.81	984.50
		Gap (%)	3.44	317.76	81.68	67.78	0.00
		Time (s)	125.19	124.20	11008.29	182.07	122.70
	I+F+T	Makespan	1059.00	4287.00	1899.50	1748.88	1011.20
		Gap (%)	4.73	323.95	87.85	72.95	0.00
		Time (s)	125.50	124.55	11706.34	182.71	123.20

This experiment evaluates transfer across dataset families. The public benchmark families differ from the synthetic training pool in operation counts, machine eligibility density, and routing flexibility, so they test whether the dual-graph state abstraction is tied to the synthetic generator or captures reusable scheduling structure. Policies trained on 10×5 synthetic instances are directly evaluated on Barnes, Brandimarte, Dauzere, and Fattahi. As shown in Fig. 5, E2G-MoE attains median gaps of 16.13%, 7.68%, 4.70%, and 18.51%, ranking first on Brandimarte, second on Barnes and Dauzere, and third on Fattahi among learning-based methods.

The variation in gaps across benchmark families indicates that routing flexibility and eligibility sparsity still affect transfer, but E2G-MoE remains competitive among learned dispatchers without benchmark-specific fine-tuning.

D. Robustness Across Disturbance Regimes

The dynamic experiments evaluate the same trained policy under three increasingly complex disturbance regimes: I , $I + F$, and $I + F + T$. Table V therefore reports cross-scale transfer and robustness across the evaluated disturbance regimes.

Table V compares DRL policies under I , $I + F$, and $I + F + T$. E2G-MoE achieves the lowest mean makespan among the evaluated learning-based baselines in all 12 scale-regime settings, with reductions of 0.79%–7.73% relative to the strongest competing baseline. Notably, the improvement is relatively modest at the 10×5 training scale but becomes more pronounced on the larger zero-shot test scales. Under $I + F + T$, the reductions are 2.01%, 7.73%, 5.94%, and 4.51% at 10×5 , 30×10 , 50×10 , and 100×25 , respectively. This pattern provides further evidence of favorable cross-scale generalization, with more pronounced improvements on the larger tested scales than at the training scale.

On public benchmarks under $I + F + T$, Fig. 6 reports E2G-MoE median gaps of 15.00%, 0.41%, 0.00%, and 14.68%. E2G-MoE achieves the lowest median gap on all four benchmarks among the evaluated learning-based methods and an overall mean gap of 10.24%. This pattern is consistent with the proposed design but does not establish robustness beyond the evaluated settings.

E. Ablation Study

Table VI reports a common family of controlled ablations. The single-graph and SGSR variants isolate graph decomposition. On top of SGSR, PM-Mono replaces the expert bank and router with an approximately parameter-matched monolithic actor, CA-HMoE removes the disturbance descriptor from routing, and RR-HMoE replaces learned routing with a random routing rule. All actor variants are trained from scratch under the same PPO and I+F+T evaluation protocol.

TABLE VI
CONTROLLED ABLATION RESULTS UNDER THE $I + F + T$ REGIME.

Variant	Makespan		Gap (%)		Time (s)	
	10×5	50×10	10×5	50×10	10×5	50×10
Single graph	674.54	2962.12	70.16	96.60	0.52	6.21
SGSR	451.90	1817.57	14.00	20.63	0.49	6.28
SGSR + PM-Mono	573.64	2658.11	44.71	76.42	0.67	8.31
SGSR + CA-HMoE	494.62	2046.15	24.77	35.80	0.51	6.32
SGSR + RR-HMoE	469.53	1843.83	18.44	22.38	0.52	6.31
Full HMoE	396.42	1506.70	0.00	0.00	0.52	6.31

SGSR reduces makespan relative to the single-graph variant by 33.0% on 10×5 and 38.6% on 50×10 . Full HMoE achieves lower makespans than PM-Mono, CA-HMoE, and RR-HMoE while maintaining inference times comparable to those of other HMoE variants. These controlled comparisons indicate that the observed gain cannot be explained by actor capacity or expert multiplicity alone and support learned disturbance-conditioned routing as an important contributor in the evaluated settings.

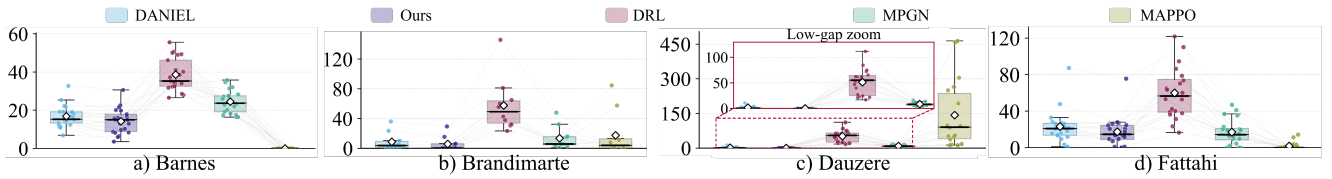


Fig. 6. Dynamic performance distributions under the I+F+T regime on the four public benchmark families.

F. Expert Routing Analysis

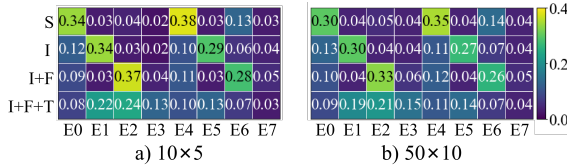


Fig. 7. Average routing weights of the eight experts across the evaluated disturbance regimes at the 10×5 and 50×10 scales.

The averaged routing weights in Fig. 7 exhibit regime-dependent patterns. Routing entropy ranges from 0.58 to 0.67, and the Jensen–Shannon divergence from S ranges from 0.092 to 0.172. Because the routed expert outputs determine candidate-pair logits, the observed routing shifts correspond to different mixtures of expert transformations rather than usage-frequency changes alone. Together with Table VI, these results provide evidence that learned routing adapts expert allocation across the evaluated disturbance regimes.

VI. CONCLUSION

This paper proposes E2G-MoE for DFJSP under concurrent disturbances. It combines an event-consistent decision interface, a dual-graph encoder separating stable precedence from dynamic competition, and a disturbance-conditioned HMoE actor. Within the evaluated synthetic instances and public benchmarks, the results provide evidence of competitive performance among the considered learning-based dispatchers, robustness in the tested scale–regime settings, and transfer to larger tested scales and benchmark families without fine-tuning. The current framework employs a relatively rich state representation, while the minimal information required for effective scheduling remains unclear. Future work will investigate compact scheduling representations that preserve performance with substantially fewer features, and explore the integration of scheduling priors and multi-agent coordination for more efficient and scalable decision making.

REFERENCES

- [1] J. Bi, R. Wu, H. Yuan, Z. Wang, J. Zhang, and M. Zhou, “Ontology-Based Semantic Reasoning for Multisource Heterogeneous Industrial Devices Using OPC UA,” *IEEE Internet of Things Journal*, vol. 12, no. 13, pp. 25020–25032, Jul. 2025.
- [2] C. Ngwu, Y. Liu, and R. Wu, “Reinforcement learning in dynamic job shop scheduling: A comprehensive review of AI-driven approaches in modern manufacturing,” *Journal of Intelligent Manufacturing*, vol. 37, no. 3, pp. 1093–1108, Mar. 2025.
- [3] J. Bi, C. Wang, Z. Wang, J. Zhang, H. Yuan, and J. Zhang, “Dual-GNN-driven cooperative optimization for makespan-minimized and large-scale 3C dynamic job-shop scheduling,” *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 4350–4364, Feb. 2026.
- [4] S. Luo, “Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning,” *Applied Soft Computing*, vol. 91, Art. no. 106208, Jun. 2020.
- [5] W. Song, X. Chen, Q. Li, and Z. Cao, “Flexible job-shop scheduling via graph neural network and deep reinforcement learning,” *IEEE*
- [6] M. Zhang, L. Wang, F. Qiu, and X. Liu, “Dynamic scheduling for flexible job shop with insufficient transportation resources via graph neural network and deep reinforcement learning,” *Computers & Industrial Engineering*, vol. 186, Art. no. 109718, Dec. 2023.
- [7] Z. Wang and W. Liao, “Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning,” *Journal of Intelligent Manufacturing*, vol. 35, no. 6, pp. 2593–2610, Jun. 2024.
- [8] I. G. Smit, J. Zhou, R. Reijnen, Y. Wu, J. Chen, C. Zhang, Z. Bukhsh, Y. Zhang, and W. Nuijten, “Graph neural networks for job shop scheduling problems: A survey,” *Computers & Operations Research*, vol. 176, Art. no. 106914, 2025.
- [9] A. Hendawy, J. Peters, and C. D’Eramo, “Multi-task reinforcement learning with mixture of orthogonal experts,” arXiv preprint arXiv:2311.11385, 2023.
- [10] T. Willi, J. Obando-Ceron, and J. Foerster, “Mixture of experts in a mixture of reinforcement learning settings,” arXiv preprint arXiv:2406.18420, 2024.
- [11] G. Cheng, L. Dong, W. Cai, and C. Sun, “Multi-task reinforcement learning with attention-based mixture of experts,” *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3812–3819, Jun. 2023.
- [12] R. Wang, G. Wang, J. Sun, F. Deng, and J. Chen, “Flexible job shop scheduling via dual attention network-based reinforcement learning,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 3, pp. 3091–3102, Mar. 2024.
- [13] L. Wan, X. Cui, H. Zhao, L. Fu, and C. Li, “A novel method for solving dynamic flexible job-shop scheduling problem via DIFFormer and deep reinforcement learning,” *Computers & Industrial Engineering*, vol. 198, Art. no. 110688, Dec. 2024.
- [14] R. Wang, Y. Jing, C. Gu, et al., “End-to-end multitarget flexible job shop scheduling with deep reinforcement learning,” *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 4420–4434, 2025.
- [15] K. Lei, P. Guo, W. Zhao, et al., “A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem,” *Expert Systems with Applications*, vol. 205, Art. no. 117796, 2022.
- [16] H. Xu, J. Zheng, L. Huang, J. Tao, and C. Zhang, “Solving dynamic multi-objective flexible job shop scheduling problems using a dual-level integrated deep Q-network approach,” *Processes*, vol. 13, no. 2, Art. no. 386, Jan. 2025.
- [17] H. Yuan, Q. Hu, J. Bi, G. Gong, J. Zhang, and M. C. Zhou, “Machine-level collaborative manufacturing and scheduling for heterogeneous plants,” *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 16591–16603, May 2024.
- [18] C. Duan, Y. Mo, and Z. Zhang, “Deep reinforcement learning for event-driven predictive–reactive multi-objective scheduling in dynamic flexible job shop,” *Journal of Manufacturing Systems*, vol. 83, pp. 992–1008, Dec. 2025.
- [19] M. Xu, Y. Mei, F. Zhang, and M. Zhang, “Learn to optimise for job shop scheduling: A survey with comparison between genetic programming and reinforcement learning,” *Artificial Intelligence Review*, vol. 58, no. 6, Art. no. 160, Mar. 2025.
- [20] N. Marques, G. Figueira, and L. Guimaraes, “Dynamic dispatching rule selection for the job shop scheduling problem,” *Computers & Industrial Engineering*, vol. 210, Art. no. 111471, Dec. 2025.
- [21] J. Ding, M. Chen, T. Wang, J. Zhou, X. Fu, and K. Li, “A survey of AI-enabled dynamic manufacturing scheduling: From directed heuristics to autonomous learning,” *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–36, 2023.
- [22] C. Zhang, W. Song, and Z. Cao, “Learning to dispatch for job shop scheduling via deep reinforcement learning,” in *Proceedings of the Advances in Neural Information Processing Systems*, vol. 33, pp. 1621–1632, Dec. 2020.
- [23] K. Lei, P. Guo, Y. Wang, J. Zhang, X. Meng, and L. Qian, “Large-scale dynamic scheduling for flexible job-shop with random arrivals of new jobs by hierarchical reinforcement learning,” *IEEE Transactions on Industrial Informatics*, vol. 20, no. 1, pp. 1007–1018, Jan. 2024.

- [25] T. Gabel and M. Riedmiller, "Distributed policy search reinforcement learning for job-shop scheduling tasks," *International Journal of Production Research*, vol. 50, no. 1, pp. 41–61, 2012.
- [26] Y. Zhang, H. Zhu, D. Tang, T. Zhou, and Y. Gui, "Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems," *Robotics and Computer-Integrated Manufacturing*, vol. 78, Art. no. 102412, Dec. 2022.
- [27] R. Liu, R. Piplani, and C. Toro, "A deep multi-agent reinforcement learning approach to solve dynamic job shop scheduling problem," *Computers & Operations Research*, vol. 159, Art. no. 106294, Nov. 2023.
- [28] L. Zhang, Y. Yan, C. Yang, and Y. Hu, "Dynamic flexible job-shop scheduling by multi-agent reinforcement learning with reward-shaping," *Advanced Engineering Informatics*, vol. 62, Art. no. 102872, Oct. 2024.



Hongyao Ma (Graduate Student Member, IEEE) received the B.S. and M.S. degrees from Shandong Jiaotong University, China, in 2021 and 2024, respectively. He is currently pursuing the Ph.D. degree with Beijing University of Technology, Beijing, China. His research interests include intelligent perception and optimization scheduling for large-scale complex systems, reinforcement learning, intelligent optimization, and dynamic scheduling. He received the Best Student Paper Award at the 2026 IEEE International Conference on Cybernetics.

tional Conference on Cybernetics.



Jing Bi (Senior Member, IEEE) received the B.S. and Ph.D. degrees in computer science from Northeastern University, Shenyang, China, in 2003 and 2011, respectively. From 2013 to 2015, she was a Post-Doctoral Researcher with the Department of Automation, Tsinghua University, Beijing, China. From 2018 to 2019, she was a Visiting Research Scholar with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ, USA. She is currently a Professor with the

College of Computer Science, Beijing University of Technology, Beijing, China. She has over 200 publications in international journals and conference proceedings. Her research interests include distributed computing, cloud and edge computing, large-scale data analytics, machine learning, industrial Internet, and performance optimization. She is an Associate Editor of the *IEEE Transactions on Systems, Man, and Cybernetics: Systems* and *IEEE Transactions on Industrial Informatics*.



Ziqi Wang (Graduate Student Member, IEEE) is currently pursuing the Ph.D. degree with the School of Software Technology, Zhejiang University, Hangzhou, China. Before that, he received the B.S. degree in Internet of Things in 2022 and the M.S. degree in Software Engineering in 2025, both from Beijing University of Technology, China. His research interests include mobile edge computing, task scheduling, intelligent optimization algorithms, and deep learning. He received the Best Paper Award in 2024 ICAIS & ISAS and the Best Application Paper Award in the 21st IEEE ICNSC.

ISAS and the Best Application Paper Award in the 21st IEEE ICNSC.



Jiahui Zhai (Member, IEEE) received the B.S. degree in Software Engineering from Zhengzhou University, Henan, China, in 2019, the M.S. degree and Ph.D. degree in Software Engineering from Beijing University of Technology, Beijing, China, in 2022 and 2026. From 2024 to 2025, he visited University College Dublin, Ireland, as a visiting scholar. His current research interests include complex intelligent systems, intelligent perception and control, and distributed machine learning. He was the recipient of the Best Paper Award-Finalist at the 18th IEEE International Conference on Networking, Sensing and Control (ICNSC). He received the Best Paper in Technology from the IEEE Robotics & Automation Society.

recipient of the Best Paper Award-Finalist at the 18th IEEE International Conference on Networking, Sensing and Control (ICNSC). He received the Best Paper in Technology from the IEEE Robotics & Automation Society.



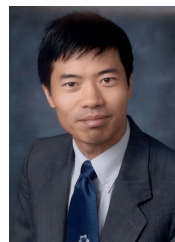
Junqi Zhang received the B.S. and Ph.D. degrees from the Department of Automation, Tsinghua University, Beijing, China, in 2015 and 2021, respectively. From 2021 to 2022, he was an Algorithm Engineer with JD.com, focusing on advertisement retrieval. From 2022 to 2023, he was with Qiyuan Laboratory as a Researcher, focusing on vision-language models. His research interests include information retrieval, recommender systems, multimodal models, self-supervised learning, and explainable artificial

intelligence.



Haitao Yuan (Senior Member, IEEE) received the Ph.D. degree in computer engineering from New Jersey Institute of Technology (NJIT), Newark, NJ, USA, in 2020. He is currently the Deputy Director with the Department of Science and Technology Innovation, Wenchang International Aerospace City, Hainan, China. He is also an Associate Professor with the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China. He is named in the world's top 2% of scientists list.

His research interests include the Internet of Things, edge computing, deep learning, data-driven optimization, and computational intelligence algorithms. He received the Chinese Government Award for Outstanding Self-Financed Students Abroad, the 2021 Hashimoto Prize from NJIT, the Best Paper Award in the 17th IEEE International Conference on Networking, Sensing and Control (ICNSC), and the Best Student Paper Award Nominee in the 2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC). He is an Associate Editor of the *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *IEEE Internet of Things Journal*, and *Expert Systems With Applications*.



MengChu Zhou (Fellow, IEEE) received the Ph.D. degree from Rensselaer Polytechnic Institute, Troy, NY, USA, in 1990. He then joined New Jersey Institute of Technology, Newark, NJ, USA, where he has been a Distinguished Professor since 2013. His research interests include Petri nets, automation, robotics, big data, the Internet of Things, cloud/edge computing, and artificial intelligence. He has authored more than 1400 publications, including 20 books, over 900 journal papers (including more than 700 IEEE Transactions papers), 33 patents, and 34 book chapters. He is a Fellow of IFAC, AAAS, CAA, and NAI.

Transactions papers), 33 patents, and 34 book chapters. He is a Fellow of IFAC, AAAS, CAA, and NAI.